

Layer 2: Semantic Defense Network (SDN)

AcademIQ Security Architecture — L2 Component Documentation

The Semantic Defense Network intercepts commands that have structurally bypassed L1 GCD and evaluates them for semantic safety before passing them to the execution layer. It relies on a rigorous 5-pass normalization algorithm to resolve obfuscation statically.

Architecture

1. **Interceptor** — Entry point (`ShellInterceptor`). Uses `LinuxLDPreloadInterceptor` or `LinuxEBPFUpProbeInterceptor` structurally in production; simulated in development via `DevelopmentShellInterceptor`.
2. **Parser** — Uses Python's `bashlex` library to tokenize POSIX shell commands safely into an AST without executing any sub-process.
3. **Five-Pass Normalizer** — Resolves all known obfuscation layers in strict sequential order (see sub-steps below).
4. **Policy Matcher** — Evaluates the resulting `CanonicalCommand` against rules defined in `config/policies/shell.yaml` using a fast allow/deny evaluator.
5. **TOCTOU Resolver** — Records the `inode` and `device_id` of target paths using `os.stat()` at decision time to prevent time-of-check-to-time-of-use races.
6. **Execution Gate** — Strictly enforces: `L1 ALLOW + L2 BLOCK = NO EXECUTION`.

Pass 1 — Variable Expansion: Safe, static resolution of environment variables against a predefined `SafeEnvironmentSnapshot`. Unresolvable variables fail closed.

Pass 2 — Encoding Decode: Detects and decodes HEX (`\\x72\\x6d`), OCTAL (`\\162\\155`), BASE64 (`cm0=`), and URL encoding safely — no shell execution.

Pass 3 — ANSI-C Quoting: Normalizes strings like `$'r\\x6d'` into `rm`. Handles escape sequences recursively.

Pass 4 — Alias/Function Resolution: Applies known aliases from a safe shell metadata snapshot (e.g., `ll` → `ls -la`).

Pass 5 — Canonicalization: Strict path canonicalization: resolves `../` and `/../` traversal, removes shell quoting artifacts, normalizes whitespace.

Static Validation Model

Crucially, **no command execution occurs during normalization**. Command substitutions such as subshell invocations are marked as unresolved. In strict mode, unresolved substitutions trigger a hard **BLOCK** to fail closed, preventing any ambiguous command from reaching the execution layer.

Platform Limitations (Windows)

- **TOCTOU** — Linux can use `O_PATH` for robust lockless resolution. On Windows, we rely on checking `st_ino` (`FileIndex`) and `st_dev` immediately before execution.
- **Paths** — The canonicalizer ensures cross-platform consistency by mapping `\\` to `/` internally after `realpath` resolution. Native `LD_PRELOAD` interceptors are structurally mocked on Windows.

- **eBPF Uprobe** — `LinuxEBPFUprobeInterceptor` raises `NotImplementedError` on non-Linux platforms. Use `DevelopmentShellInterceptor` for local development and testing.

Event Schema

L2 emits `NormalizedCommandEvent` after processing every intercepted shell command. This event is consumed by the orchestrator and persisted to the L6 ECES evidence chain.

`event_id` — Unique identifier for this normalized event
`agent_id` — Agent session identifier (propagated from original `ToolInvocationEvent`)
`command_text` — Canonical command string after all 5 normalization passes
`original_command_hash` — SHA-256 of the raw, unmodified command string
`canonical_command_hash` — SHA-256 of the fully normalized canonical command
`normalization_passes` — Ordered list of transformation passes applied
`obfuscation_detected` — Boolean: True if any encoding/obfuscation was found and normalized
`policy_result` — Final decision: ALLOW | BLOCK | REVIEW
`matched_rule` — Identifier of the policy rule that determined the decision
`path_identities` — TOCTOU resolution locks: inode + device_id per resolved path
`security_decision` — Final security decision propagated to the orchestrator

Source: `l2_sdn/` · Policy: `config/policies/shell.yaml` · Tests: `tests/integration/test_l2_adversarial.py` (20 passing)